

Assignment 8 (30 points)

Document Object Model: Event Handling and jQuery Intro

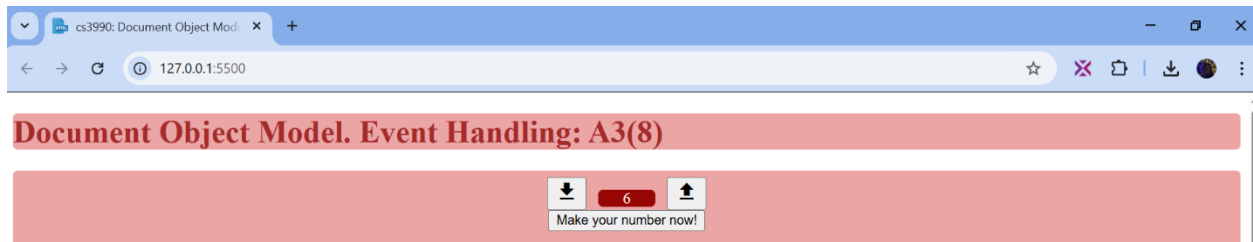
Task1 – 10 points

Develop the class *NumberGenerator* that creates and displays the component (a html block) accordingly the following template

The component contains 3 buttons:

when you click on *Generate*, random integer from 0 to 100 is displayed in the div;

Greater and Less buttons (displayed as arrows) provide the ability to change a numeric value (by 1)

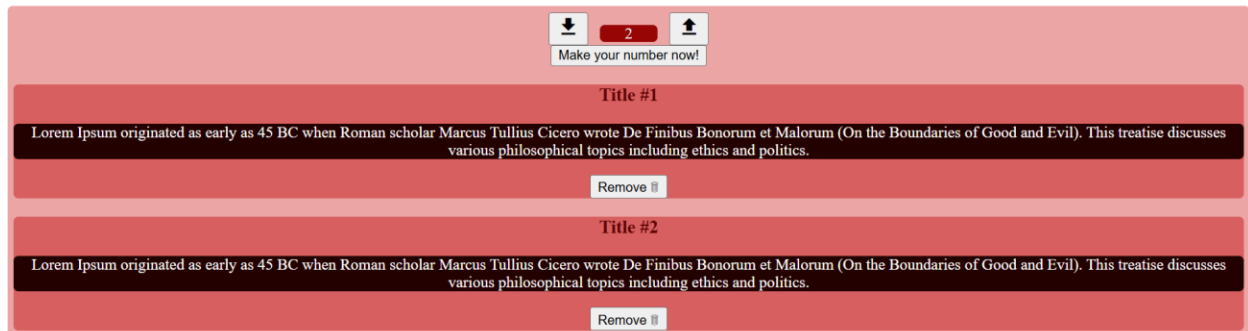


Create a function to generate a block with news. Each news item should have a "Remove" button, when clicked, the corresponding news item disappears.

The number of generated news items is equal to a random integer from the block above.

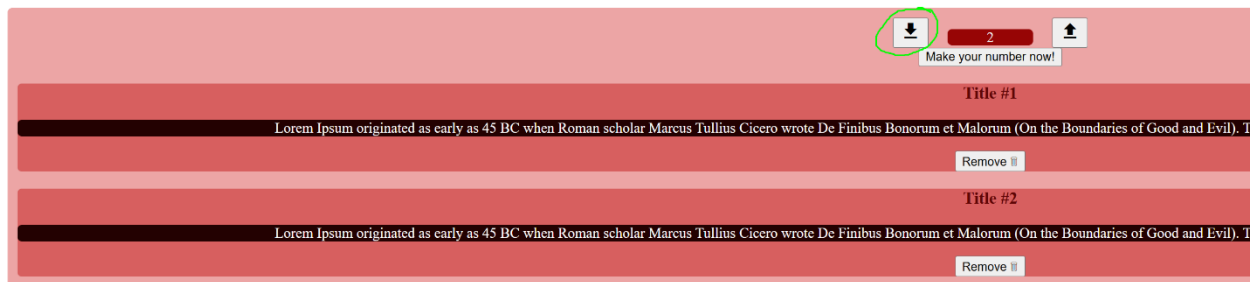


Document Object Model. Event Handling: A3(8)

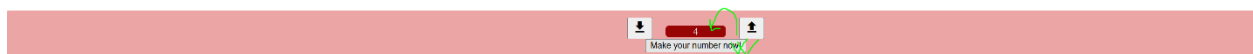
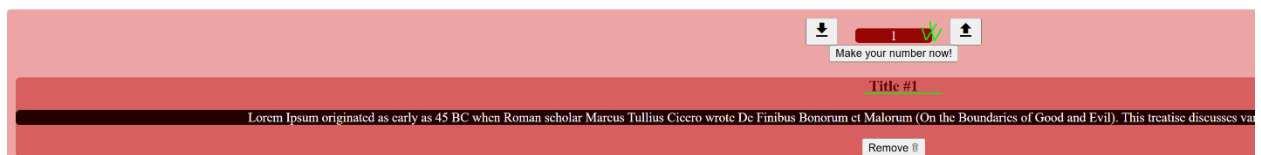


When the random number changes (by clicking on any of the 3 buttons above), the content of the news block is updated accordingly. See examples below.

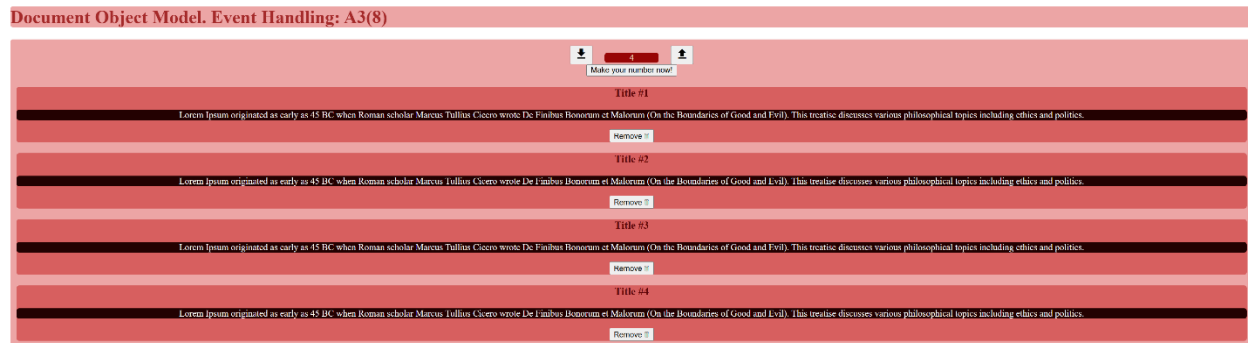
Document Object Model. Event Handling: A3(8)



Result:



Result:

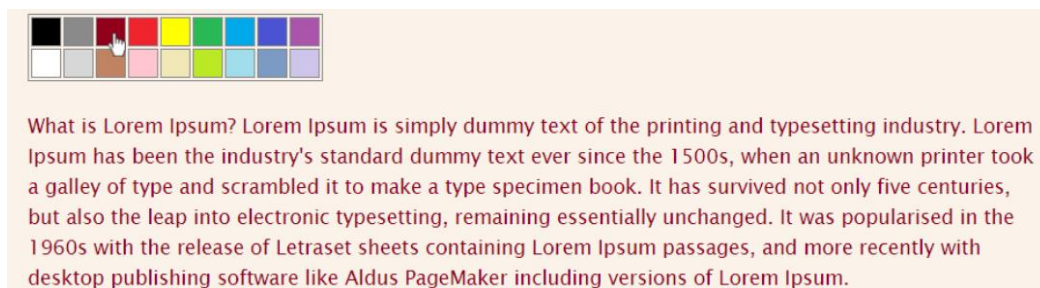


Task 2 - 5 points

Create a component (as a class) *ColorButton* with the following behavior:

- when you click on it, the font color of the block below (accessible to the event handler via its ID) changes;
- when you mouse over the it, the background color of the block below is being changed (this background color is kept while the mouse is still on this color cell)

Define a function to generate the color palette (see an example below).



Notes!

- **Event delegation** is supposed to be used for this task (instead of assigning a handler to each color block put a single handler on their common ancestor).
- **Object handlers'** approach is supposed to be used to store all handlers' logic and process the event accordingly to its type. Create a separate class *PaletteMenu* for this purpose.

Task 3 - 5 points

Improve a menu (from the lecture notes, see below) that opens and collapses on user's click

► Sweeties (click me)!

▼ Sweeties (click me)!

Cake

Donut

Honey

by adding the following features:

- ✓ when you select a menu item,
 - ✓ the text of that item turns red,
 - ✓ and an image of that type of sweet appears under the list of sweeties (menu).
- ✓ when selecting another item, the red font color “moves” to it (“is removed” from the previous one) and another (corresponding) image is displayed under the list of sweeties;
- ✓ when the menu collapses, the image disappears and the red font color for the menu items disappears too.

Task 4 - 10 points

You have the following html-file:

```
<body>
  <div id="colors">
    <h2> Select color</h2>
  </div>

  <div id="fruits">
    <ul>Fruits:</ul>
  </div>
</body>
```

And the list of fruits with their colors (stored as an array of objects, see an example below)

```
let hannaFruits=[
  { fruit:"apple",
    color:"red"
  },
  { fruit:"pear",
    color:"green"
  },
  { fruit:"mango",
    color:"red"
  },
  { fruit:"plum",
    color:"blue"
  }
]
```

Create a class *Fruit* to fill the list (from your html-file) with items. The fruit's name and color are supposed to be used in the class constructor. Define the separate method (e.g. *show()*) to generate the list's item and display it.

Create a class *btnColor* to generate a collection of buttons accordingly to existing fruits' colors. The color name is supposed to be used in the class constructor to

specify the button's label and its background color. Define the separate method (e.g. *show()*) to generate a button and display it within the following block:

```
<div id="colors">
  <h2> Select color</h2>
</div>
```

See an example of result (based on *hannaFruits* data) below:

Select color

red green blue

Fruits:

- apple
- pear
- mango
- plum

By using **jQuery** and **Event delegation** approach add the following behavior for the buttons: when clicking on the button, the fruits of the corresponding color must be highlighted with a shadow (see an example below)

Select color

red green blue

Fruits:

- apple
- pear
- mango
- plum

Note!!! instead of assigning a handler to each button put a single handler on their common ancestor

Create a derived class *RatedFruit* with the ability to rate this item. The initial rating is provided as an additional parameter for the class constructor. Override the *show()* method to display the star rating as follows:

Star Rating: ★★☆☆☆

By using jQuery and Event delegation approach add the following behavior for the stars: if you click with a mouse on any of the stars, all the stars from the first to the one you have clicked on recolor to orange, and the rest turn black.

Hint 😊: use jQuery *prev()* method (<https://api.jquery.com/prev/>).